

HRMS

Human Resources Management System

Software Requirements Specification

Architecture & Design Documentation

Version 1.0 | May 2025

Spring Boot · Spring Cloud · React.js · Docker

1. Introduction

The Human Resources Management System (HRMS) is a distributed, microservices-based web application designed to automate and streamline core human resource operations. Built with Spring Boot and Spring Cloud, the platform delivers secure, scalable management of employees, departments, payroll processing, leave requests, and attendance tracking.

1.1 Purpose

This document provides the full Software Requirements Specification (SRS), architecture overview, design patterns, OCL constraints, and UML diagrams for the HRMS project. The system is intended for:

- HR departments managing employee lifecycles
- Company administrators overseeing the full platform
- Department managers handling team approvals and records
- Employees accessing their own profiles, payslips, and leave status

1.2 Scope

The HRMS is a complete distributed HR platform that separates authentication, employee data, department management, payroll, and leave/attendance into independent Spring Boot microservices. All services communicate through a central API Gateway and register dynamically via Eureka Service Discovery.

1.3 Technology Stack

Layer	Technology
Backend Framework	Spring Boot 3.x
Microservices	Spring Cloud (Eureka, Gateway, Config)
Security	Spring Security + JWT (JSON Web Tokens)
Database	PostgreSQL / MySQL (per-service)
Frontend	React.js
Deployment	Docker & Docker Compose
Cross-Cutting	AOP (Logging, Auditing, Transactions)

2. System Users & Roles

The HRMS supports four distinct user roles, each with a scoped set of permissions enforced at the API layer through JWT-based role-based access control.

Role	Responsibilities	Access Level
Admin	Full system control, user management, all services	All modules
HR Manager	Employee CRUD, payroll generation, leave oversight	Employee, Payroll, Leave
Department Manager	Team management, leave approvals, attendance	Department, Leave, Attendance
Employee	View own profile, submit leave, view payslips	Read-only own data

3. Functional Requirements

3.1 Authentication & Authorization

- Users shall register and log into the system with email and password.
- The system shall generate JWT tokens on successful authentication.
- JWT tokens shall encode the user's role and be validated on every API request.
- Unauthorized users shall receive HTTP 401/403 responses on protected resources.

3.2 Employee Management

- Admins and HR Managers shall perform full CRUD operations on employee records.
- Employees shall view and update limited personal profile information.
- Employee records shall store: name, email, position, salary, department, and attendance info.

3.3 Department Management

- Admins shall create, update, and delete department records.
- Department Managers shall manage employees within their assigned department.
- Each department shall have an assigned manager and tracked headcount and budget.

3.4 Payroll Management

- The system shall calculate net salary from base salary, bonuses, and deductions.
- HR Managers shall generate and store monthly payroll records and payslips.
- Employees shall view their own payroll history and current payslip.

3.5 Leave Management

- Employees shall submit leave requests with type, date range, and reason.
- Department Managers and HR Managers shall approve or reject leave requests.
- The system shall maintain a full leave history with status tracking (PENDING / APPROVED / REJECTED).

3.6 Attendance Management

- The system shall record and track daily attendance per employee.

- Attendance records shall store date, status (Present / Absent / Late), and mode (On-site / Remote).
- HR Managers shall access attendance reports and statistics.

3.7 Audit Logging

- All significant API operations shall be logged using AOP (Aspect-Oriented Programming).
- Audit logs shall capture actor, operation, and timestamp for debugging and compliance.

3.8 API Gateway & Service Discovery

- All client requests shall be routed through the API Gateway.
- Services shall register and discover each other dynamically via Eureka.
- Centralized configuration shall be delivered through the Config Server.

4. Non-Functional Requirements

Category	Requirement
Security	JWT auth on all APIs; passwords BCrypt-hashed; RBAC enforced at gateway.
Performance	APIs should respond within 500ms under normal load; support concurrent users.
Scalability	Independent service scaling; new services added without affecting existing modules.
Maintainability	Layered architecture; single-responsibility classes; centralized configuration.
Reliability	Services recover after failures; config server ensures stable deployments.
Usability	Responsive React.js frontend; clear navigation between all modules.
Portability	Docker Compose deployment across development and production environments.

5. System Architecture

The HRMS follows a Microservices Architecture where each business domain is an independent Spring Boot service. Services communicate via REST APIs, routing all traffic through the API Gateway. Service discovery is handled dynamically by Eureka, and shared configuration is delivered by the Config Server.

5.1 Microservice Responsibilities

Service	Responsibility
Auth Service	Registration, login, JWT issuance, role validation
Employee Service	Employee CRUD, contracts, personal data management
Department Service	Department records, manager assignment, structure
Payroll Service	Salary computation, bonuses, deductions, payslips
Leave & Attendance Service	Leave workflow, approval routing, attendance tracking
API Gateway	Single entry point for all frontend requests
Eureka Server	Dynamic service registration and discovery
Config Server	Centralized configuration for all services

5.2 Cross-Cutting Concerns

- Logging — AOP-based request and response logging across all services
- Security Auditing — AOP aspects capture actor, endpoint, and timestamp
- Transaction Management — @Transactional boundaries in service classes

5.3 Deployment Model

Each service runs in its own Docker container. Docker Compose orchestrates the full stack locally. The API Gateway is the sole public-facing entry point. Each service has its own database schema, supporting full microservice isolation.

6. Design Patterns

6.1 Layered Architecture

All microservices follow a strict four-layer structure: Controller → Service → Repository → Entity. Controllers accept HTTP requests. Services contain business rules. Repositories isolate data access. Entities model persistence without UI concerns.

6.2 Repository Pattern

Spring Data JPA repositories encapsulate all persistence logic, providing a clear data-access boundary per service.

- `UserAccountRepository` — auth-service
- `EmployeeRepository` — employee-service
- `DepartmentRepository` — department-service
- `PayrollRepository` — payroll-service
- `LeaveRequestRepository` — leave-attendance-service

6.3 DTO (Data Transfer Object) Pattern

Request and response records (DTOs) separate API payloads from persistence entities, preventing entity exposure over the wire.

- `RegisterRequest`, `LoginRequest` — auth-service
- `LeaveRequestPayload` — leave-attendance-service

6.4 Dependency Injection

All services and filters are built through constructor injection, making dependencies explicit and testable without Spring magic.

6.5 Aspect-Oriented Programming (AOP)

Cross-cutting concerns (logging, auditing) are extracted into `@Aspect` classes, keeping business methods clean.

- `RequestAuditAspect` — auth-service
- `ModuleAuditAspect` — employee-service, payroll-service

6.6 Transaction Script (Workflow Pattern)

WorkflowService in the leave-attendance-service encapsulates a complete leave approval transaction with clear pre- and post-conditions, ensuring atomic business workflows.

7. Data Model

The following entities represent the core domain objects persisted by each microservice. Relationships are enforced via foreign key references (stored as string IDs across service boundaries to preserve loose coupling).

7.1 Entities

Entity	Key Fields	Relationships
UserAccount	id, fullName, email, role, employeeId	Maps to Employee (optional)
Employee	id, employeeId, fullName, email, departmentId, position, salary	Belongs to Department; has PayrollRecords, LeaveRequests, Attendance
Department	id, name, manager, budget, headcount	Contains many Employees
PayrollRecord	id, employeeId, payMonth, baseSalary, bonus, deductions, netSalary, status	Belongs to Employee
LeaveRequest	id, employeeId, leaveType, startDate, endDate, reason, status	Submitted by Employee
AttendanceRecord	id, employeeId, attendanceDate, status, mode	Belongs to Employee

7.2 Entity Relationships

- One Department → many Employees
- One Employee → many PayrollRecords
- One Employee → many LeaveRequests
- One Employee → many AttendanceRecords
- One UserAccount → optionally maps to one Employee

8. OCL Constraints

The following Object Constraint Language (OCL) invariants define the business rules enforced on the domain model. These constraints ensure data integrity and valid state transitions.

8.1 UserAccount

```
context UserAccount inv EmailPresent:
    self.email <> null and self.email <> ''
context UserAccount inv FullNamePresent:
    self.fullName <> null and self.fullName <> ''
context UserAccount inv RoleAssigned:
    self.role <> null
context UserAccount inv PasswordPresent:
    self.passwordHash <> null and self.passwordHash <> ''
```

8.2 Employee

```
context Employee inv EmployeeIdPresent:
    self.employeeId <> null and self.employeeId <> ''
context Employee inv DepartmentAssigned:
    self.departmentId <> null and self.departmentId <> ''
context Employee inv SalaryNonNegative:
    self.salary >= 0
```

8.3 Department

```
context Department inv NamePresent:
    self.name <> null and self.name <> ''
context Department inv ManagerAssigned:
    self.manager <> null and self.manager <> ''
context Department inv BudgetNonNegative:
    self.budget >= 0
```

8.4 PayrollRecord

```
context PayrollRecord inv BaseSalaryNonNegative:
    self.baseSalary >= 0
context PayrollRecord inv BonusNonNegative:
    self.bonus >= 0
context PayrollRecord inv DeductionsNonNegative:
    self.deductions >= 0
context PayrollRecord inv NetNotGreaterThanGross:
    self.netSalary <= self.baseSalary + self.bonus
```

8.5 LeaveRequest

```
context LeaveRequest inv DateOrderValid:
    self.startDate <= self.endDate
context LeaveRequest inv StatusValid:
    self.status = 'PENDING' or self.status = 'APPROVED'
    or self.status = 'REJECTED'
```

8.6 AttendanceRecord

```
context AttendanceRecord inv EmployeeAssigned:
    self.employeeId <> null and self.employeeId <> ''
context AttendanceRecord inv AttendanceDatePresent:
    self.attendanceDate <> null
```

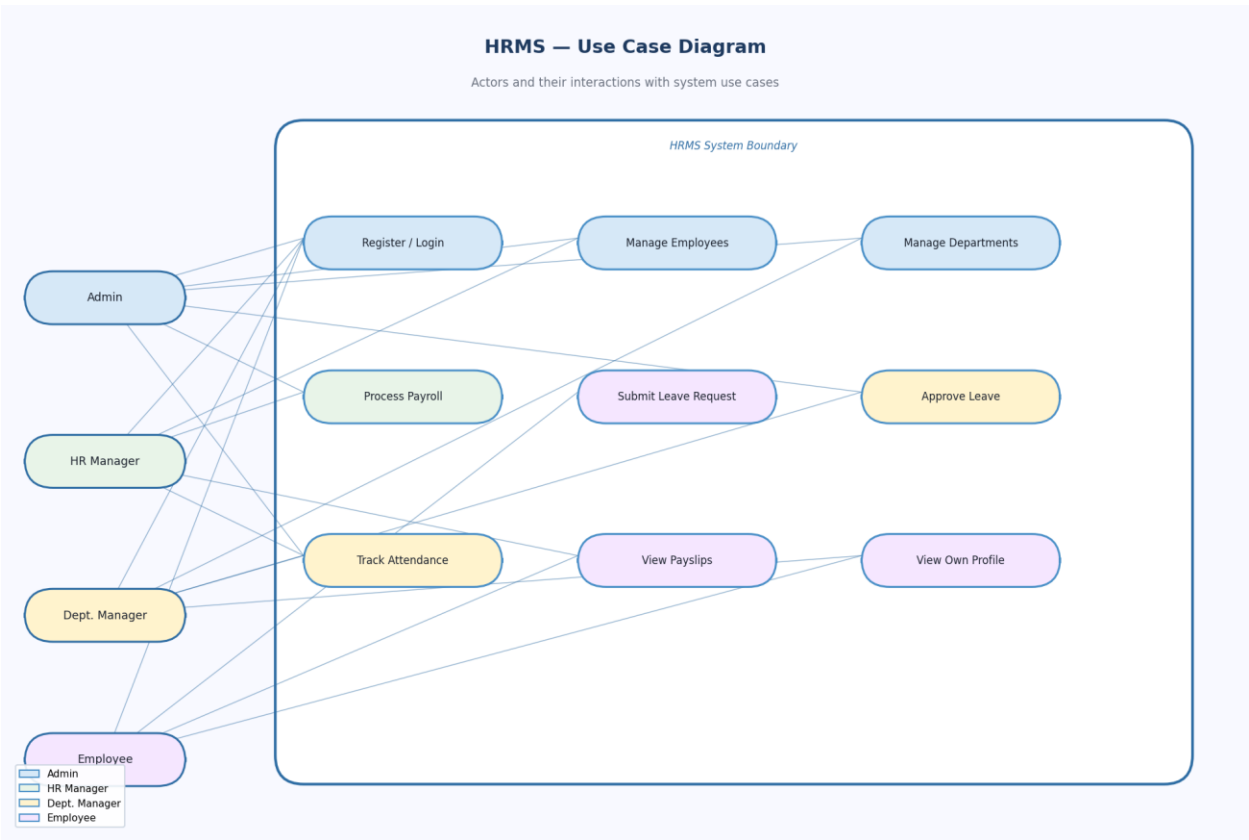
8.7 Workflow Pre-conditions

```
context WorkflowService::createLeave(payload)
    pre PayloadEmployeePresent:
        payload.employeeId <> null and payload.employeeId <> ''
context WorkflowService::resolveLeave(id, payload)
    pre DecisionStatusValid:
        payload.status = 'APPROVED' or payload.status = 'REJECTED'
```

9. UML Diagrams

9.1 Use Case Diagram

The following table maps system actors to their available use cases:

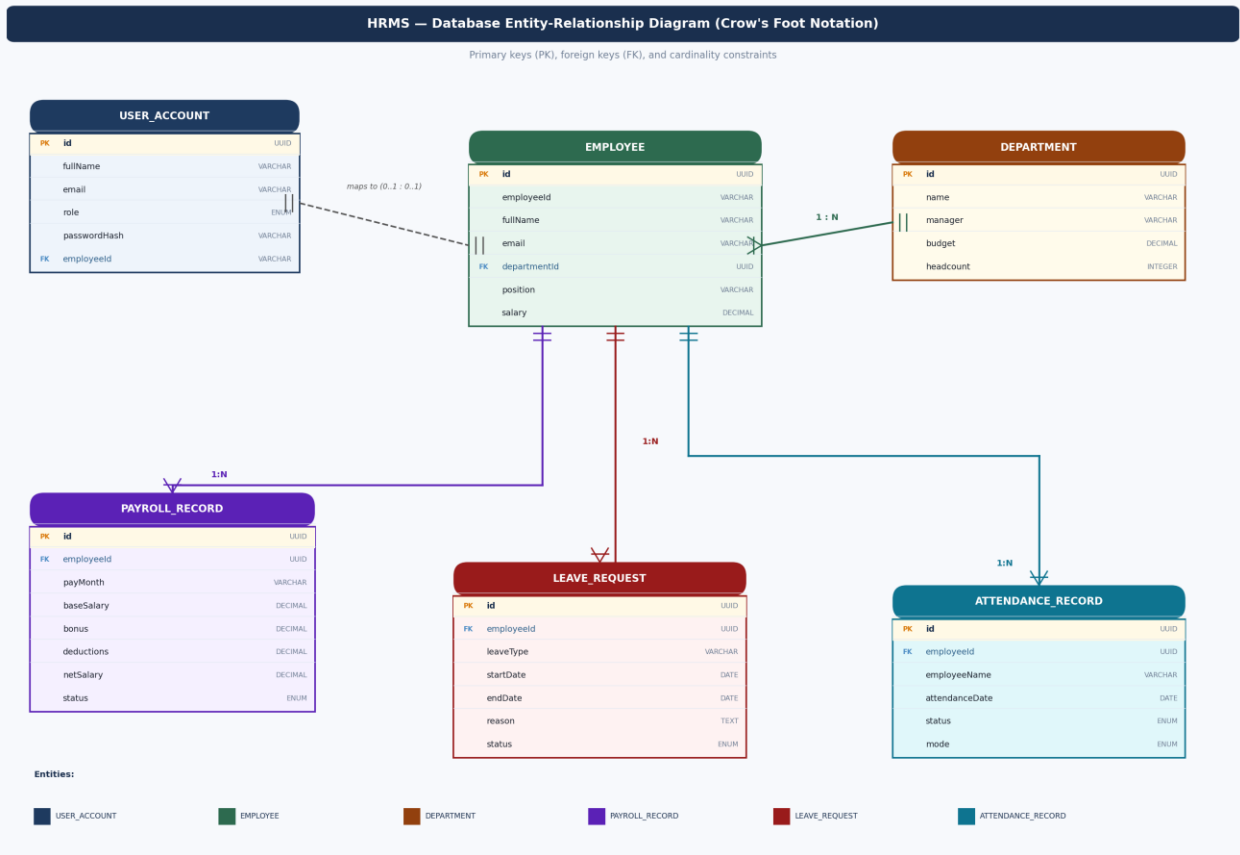


Legend: ✓ = Full Access View = Read-only — = No Access

9.2 Entity Relationship Diagram (ERD)

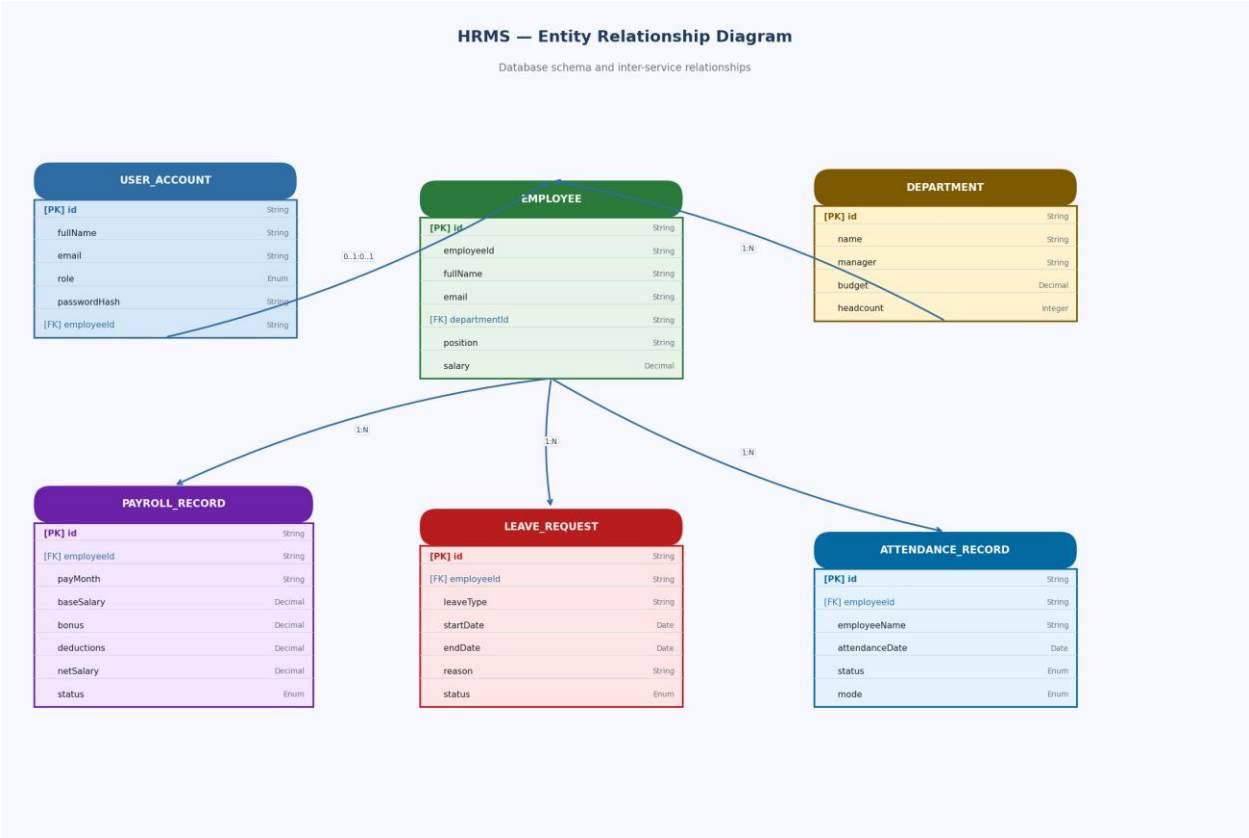
The following tables show each entity's attributes and their inter-service relationships. Relationships are maintained via employeeId and departmentId foreign key references.

Database ERD — Crow's Foot Notation



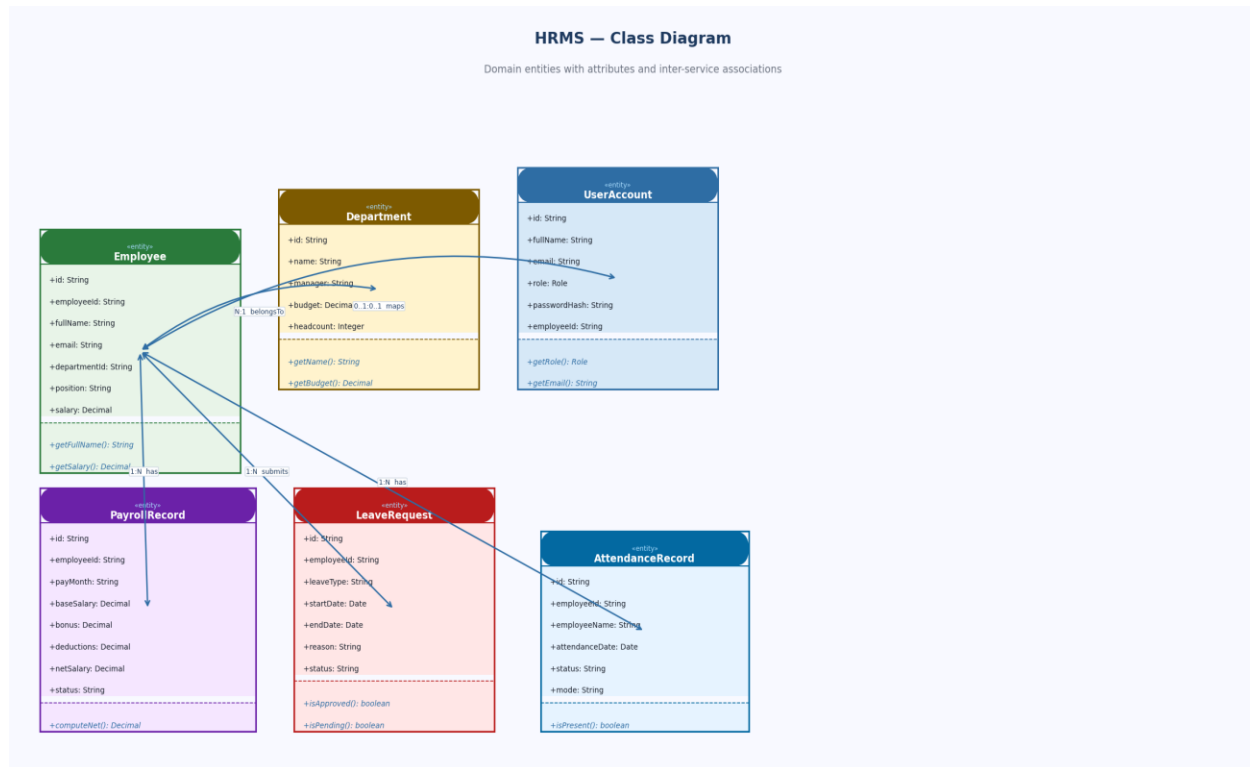
Entity Attribute Tables

USER_ACCOUNT



9.3 Class Diagram

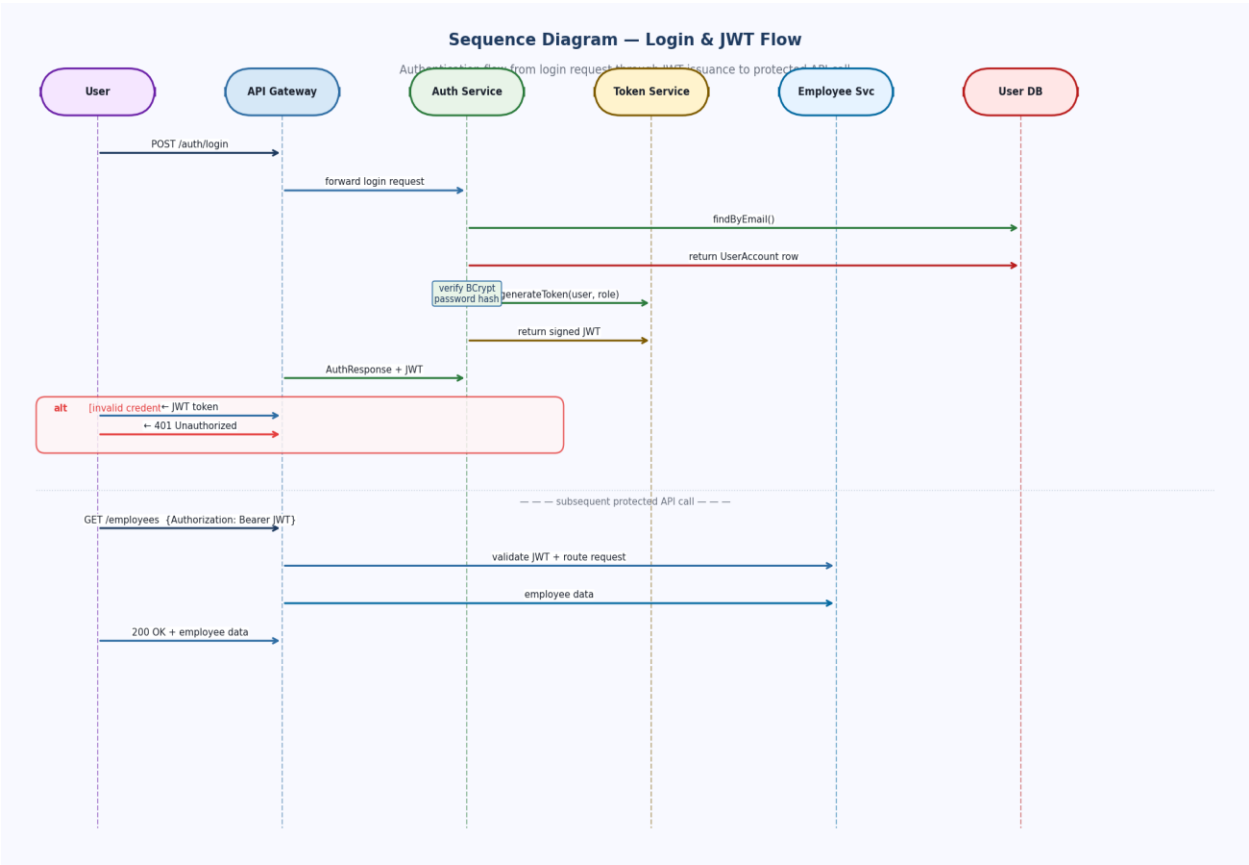
The class diagram shows the main domain entities with their key attributes and inter-service associations.



Associations: *Employee* —< *PayrollRecord* (1:N) | *Employee* —< *LeaveRequest* (1:N) | *Employee* —< *AttendanceRecord* (1:N) | *Employee* >— *Department* (N:1) | *UserAccount* —o *Employee* (optional)

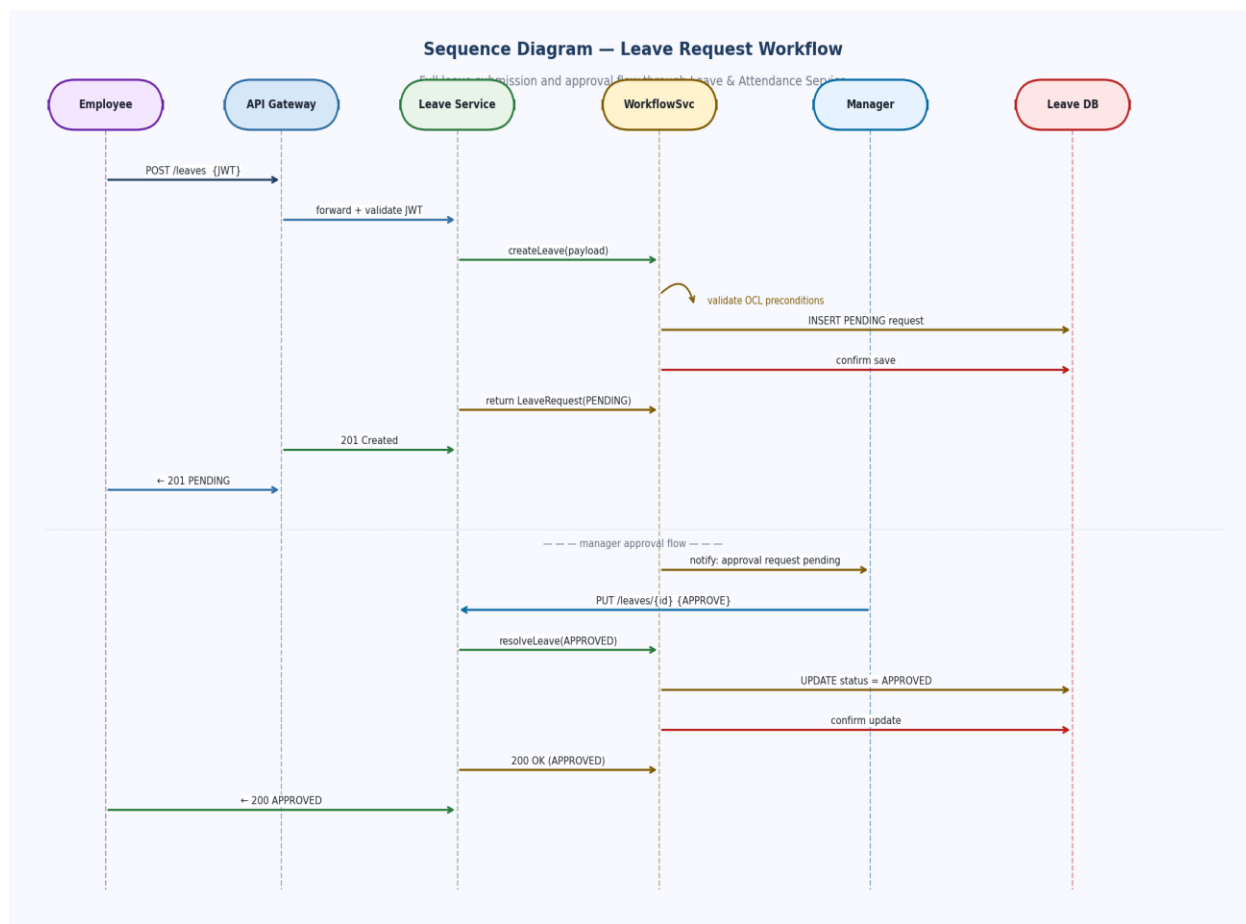
9.4 Sequence Diagram — Login & JWT Flow

This sequence illustrates the complete authentication flow: from user login request through JWT generation to a subsequent protected API call.



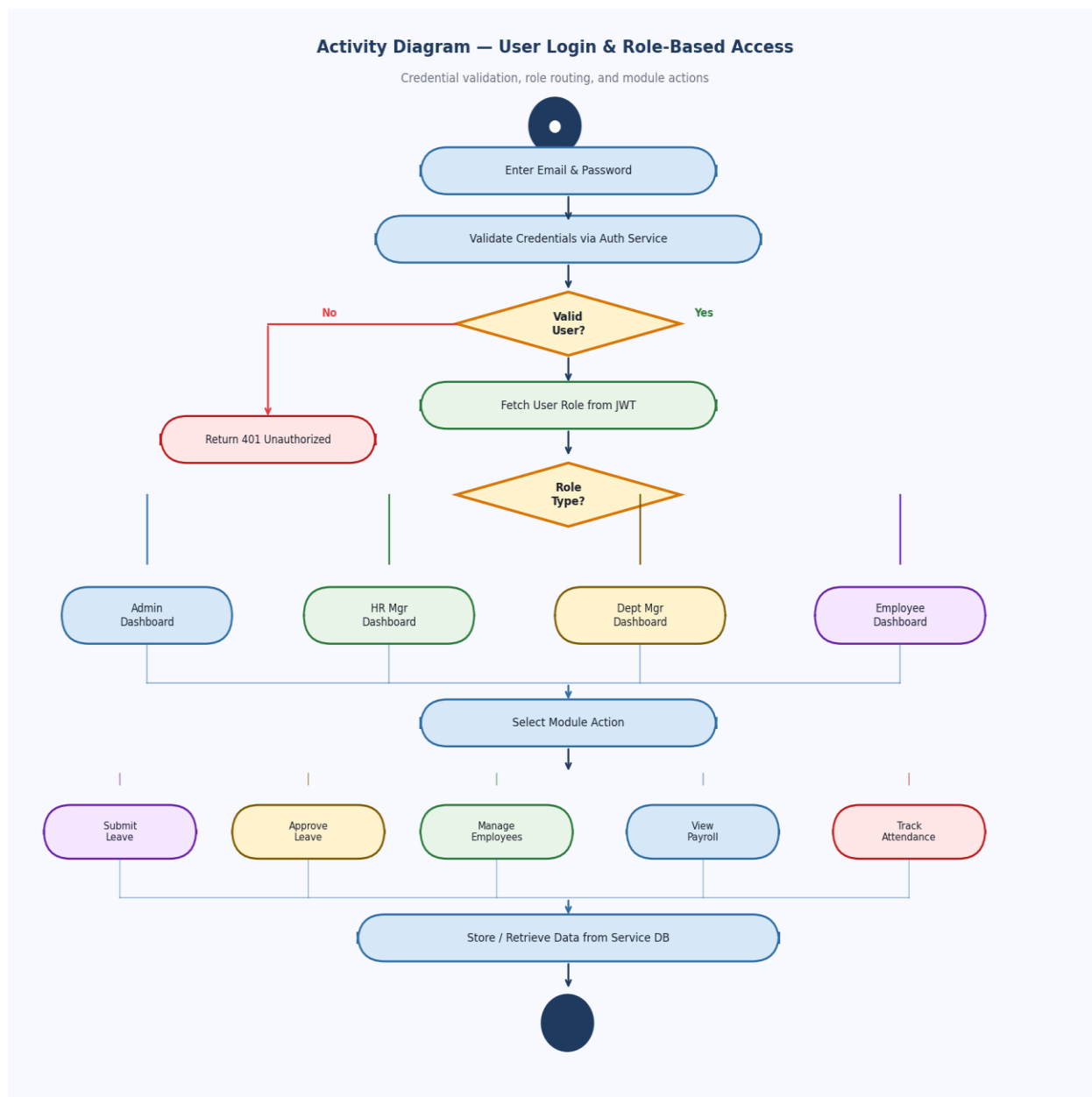
9.5 Sequence Diagram — Leave Request Workflow

This sequence shows the full leave submission and approval flow through the Leave & Attendance Service.



9.6 Activity Diagram — User Login & Role-Based Access

This activity diagram traces the login process from credential entry through role-based dashboard routing to module actions.



10. Clean Code Evidence

The codebase is structured to be readable, testable, and maintainable. The following practices are applied consistently across all five microservices.

Practice	Implementation
Single Responsibility	Each class has one reason to change — controllers route, services decide, repositories query.
Descriptive Naming	AuthService, WorkflowService, BearerTokenFilter — names communicate intent.
Constructor Injection	All dependencies declared in constructors; no field injection or hidden coupling.
DTO Separation	API payloads (RegisterRequest, LoginRequest) are separate from JPA entities.
Thin Controllers	Controllers extract request data and delegate all logic to service classes.
AOP for Cross-Cutting	Logging and auditing in @Aspect classes, never inside business methods.
Consistent Package Boundaries	controller / service / repository / entity / dto / aspect — identical across services.
Centralized Docs	All architecture, design, and requirement docs in /docs — no hidden knowledge.

11. Submission Checklist

Status	Requirement	Evidence
✓	Spring Boot Backend	backend/services/* — all 5 services
✓	REST APIs	Controller classes in each service
✓	React.js Frontend	client/ directory
✓	4+ Functional Modules	Auth, Employee, Department, Payroll, Leave+Attendance
✓	Role-Based Access	Role.java + SecurityConfig per service + JWT claims
✓	AOP Usage	RequestAuditAspect, ModuleAuditAspect in multiple services
✓	Docker Deployment	docker-compose.yml + Dockerfiles per service
✓	Spring Cloud	api-gateway, service-discovery, config-server
✓	Database	Spring Data JPA entities + repositories
✓	Auth & Registration	AuthController + AuthService + UserAccountRepository
✓	SRS Document	docs/SRS.md + Software_Document.pdf
✓	Use Case Diagram	Section 9.1 — Role × Use Case matrix
✓	Class Diagram	Section 9.3 — 6 entity classes with associations
✓	Sequence Diagrams	Sections 9.4, 9.5 — Login flow + Leave workflow
✓	Activity Diagram	Section 9.6 — Login & role routing activity
✓	ERD	Section 9.2 — Full entity tables + relationship summary
✓	OCL Constraints	Section 8 — 25+ invariants and pre-conditions
✓	Design Patterns	Section 6 — Layered, Repository, DTO, DI, AOP, Workflow
✓	Clean Code	Section 10 — 8 practices with implementation evidence

